Vula OS Group
Soko
Internet-Draft
July 2026
Intended status: Standards Track (proposal)

Trade, Routing, Attestation, Custody & Trust (TRACT)

draft-tract-00

Abstract

TRACT is an open protocol for decentralized commerce: goods, services, rentals, subscriptions, and the delivery and settlement around them, between self-sovereign identities with no marketplace operator. A seller's catalogue is a signed, content-addressed feed the seller alone controls; an offer is a claim by one seller over a product record any number of sellers may share, so a single global product view emerges from content addressing rather than from a registrar. Orders are sealed messages between the parties, never published. TRACT adopts the DMTAP substrate (identity, feeds and blobs, sync, infrastructure roles, wake) unchanged and adds the commerce spine: a four-axis offer model covering goods and services alike, buyer-side carts and multi-seller checkout, published carrier and distributor rate cards from which routing is computed locally, purchase-attested reviews with no global score, jurisdiction as a first-class field, and a payment seam that names no provider. It composes existing standards (RFC 8949 CBOR, RFC 9052 COSE, RFC 8032 Ed25519, RFC 5545 iCalendar availability, ISO 3166/4217, schema.org product vocabulary, Incoterms, GS1 identifiers where present); the novelty is the composition, not new cryptography.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of an open, royalty-free specification. Internet-Drafts are working documents; this document is a proposal and is *not* an RFC — an RFC number is assigned only upon publication by the relevant standards body. It may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use this document as reference material or to cite it other than as "work in progress."

Requirements Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals, as shown here.

Table of Contents

0. Overview & Architecture

0.1 Goals

TRACT is a protocol for **commerce between self-sovereign identities** — goods, services, rentals, subscriptions, and the delivery and settlement around them — with **no marketplace operator**, no registrar, and no token. A seller's catalogue is a signed feed the seller alone controls; a buyer's cart is state the buyer alone holds; the network in between stores nothing durable and adjudicates nothing.

Concretely, TRACT MUST provide:

1. **Sovereign selling** — a keypair is a store. No account with any platform is required to *be* a seller, and no party can delist, suspend, or edit a seller's catalogue (§1, §2).
2. **One product, many sellers** — a product record is content-addressed, so two sellers publishing the same record converge on the same address *by construction*. "Who sells this?" is a derived index anyone can rebuild, never a registry anyone runs (§2). **This is the mechanism, not yet a demonstrated solution** — independent publishers describing the same physical product do not produce identical bytes, and no deployed system achieves cross-publisher product identity without a licensed registry (§21.2).
3. **One shape for every trade** — goods, services, rentals, bookings, subscriptions, licences and made-to-order work all express as one four-axis offer, without a plugin per category (§3–§5).
4. **One cart across independent sellers** — a buyer assembles a cart spanning sellers who have never heard of each other, and checks out once (§6, §7).
5. **Delivery computed, not brokered** — carriers, distributors and peer couriers publish rate cards as public objects; routing and consolidation are computed **on the buyer's own node** from those objects, so no party sees the whole cart (§8).
6. **Settlement without a payment monopoly** — a provider-agnostic seam carries signed payment attestations; the protocol never holds funds and names no provider (§9).
7. **Trust without an authority** — reviews are signed objects proven against real purchases, and ranking is derived data any node computes. There is no global score, because computing one requires a party that aggregates and ranks (§10).
8. **Lawful by construction, worldwide** — jurisdiction, tax anchors and the responsible party are machine-readable fields on every offer and order, not an afterthought bolted onto a platform's terms of service (§11).
9. **Future-proofing** — crypto-agility and transport independence inherited from the substrate; standards reuse everywhere a standard exists (§20).

0.2 What TRACT is not

- **Not a marketplace.** There is no operator that lists sellers, ranks them, takes a cut, or can remove them. Where this document says "index" it always means derived, rebuildable data (§2.6).
- **Not a payment network.** TRACT specifies no rail, no currency, no token, and no ledger. It specifies a *seam* (§9) and the attestations that cross it.
- **Not a courier.** TRACT computes routes over published rate cards; it moves nothing.
- **Not an arbiter.** Where a dispute needs an adjudicator with power to seize funds, TRACT names the one role that may hold that power and confines it (§0.4, §9.6) rather than pretending the protocol can settle it.

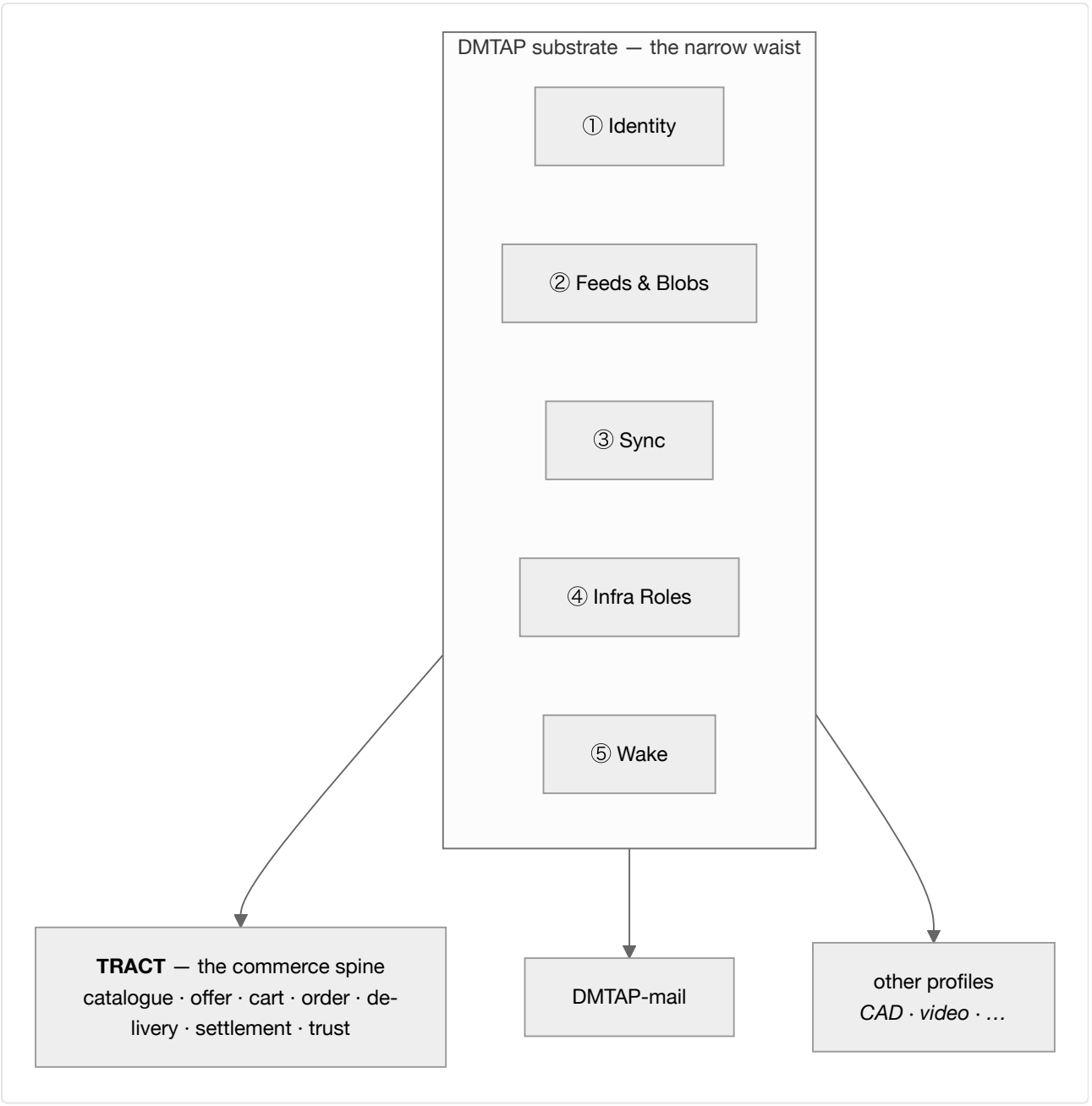
- **Not a new cryptosystem.** Every primitive is inherited from the DMTAP substrate (§0.3), which in turn profiles existing RFCs.

0.3 TRACT stands on the DMTAP substrate

TRACT is not a new stack. It **adopts** the five substrate capabilities defined in the DMTAP substrate directory, under that directory's à-la-carte adoption rule — *if a product implements a capability's function, it MUST speak that capability's spec* — and adds only the commerce spine on top.

#	Substrate capability	What TRACT uses it for
①	**Identity** — keypair `IK`, `DeviceCert`, DNS `name→key`, key transparency, 8-word key-name	seller, buyer, courier, distributor and gateway identity (§1)
②	**Feeds & Blobs** — signed append-only per-author feeds, plaintext content-addressed blobs, servable over plain HTTPS	catalogues, offers, rate cards, capacity, reviews (§2, §8, §10)
③	**Sync** — signed CRDT op algebra, range-Merkle reconciliation	carts across a buyer's devices, inventory across a seller's replicas (§6)
④	**Infrastructure Roles** — announce/resolve, signaling, circuit relay, short-TTL content-blind mailbox, cache/pin	reachability for stores and buyers behind NAT (§1.5)
⑤	**Wake** — content-free, sender-blind push	waking a sleeping seller node when an order arrives (§1.5)

Sealed order messages (§7) ride the DMTAP **MOTE** (its §2) and its message kinds. TRACT allocates no new cryptography, no new hash construction, and no new signature framing.



The consequence worth stating up front: a seller's identity, a buyer's identity, and a mail identity are the *same key*. A shopper does not create an account to buy; they already have one, and it is theirs.

0.4 Roles, and the one operator class

Like the substrate, TRACT is **roles, not node types**. Every function below is a role of the same software, taken by whoever wants it, requiring nothing rarer than a machine that is up — with exactly one exception, and confining that exception is the structural claim of this document.

0.4.1 The roles anyone may take

Role	What it does	Why anyone can run it
Seller	publishes a catalogue feed; receives sealed orders	a keypair and a box that is up
Buyer	holds a cart; sends sealed orders; computes routing	a keypair

Role	What it does	Why anyone can run it
Courier	publishes a rate card; accepts consignments	a keypair; a bicycle is enough
Distributor	publishes storage/consolidation capacity; holds goods in transit	a keypair and space
Index	builds search/aggregation over public feeds	derived data; anyone may build one, none is authoritative (§2.6)
Cache / pin	serves content-addressed objects	inherited substrate role

None of these is sold, gatekept, or registered. An index in particular is **not** a marketplace: it holds no authority, and a disagreement between an index and a feed is always resolved in favour of the feed.

> **The gap between permission and practice (§21.3).** "Any node MAY build an index" does not mean many will. A content-addressed substrate offers no global index, so discovery is the **first** function to re-centralize: whichever index becomes economically dominant becomes a de facto content-policy gatekeeper, regardless of what this document permits. That happened to the closest deployed relative of this design, and the largest live decentralized-commerce network avoided it only by adopting a central approval-gating registry (§21.4). No protocol rule here prevents it. Multiple competing indexers with verifiable completeness or censorship proofs are a candidate answer with **no deployed precedent** (§21.8). This is the weakest load-bearing claim in the document and is marked as such rather than defended.

0.4.2 The gateway — the only operator class

A **gateway** is a role like the others, with one difference that defines it: it is the **only** function in TRACT that requires **scarce resources** — a domain with TLS and uptime, and, where it settles payments, a payment-provider relationship, a money float, and jurisdiction-specific licensing. None of that can be derived from a keypair or provisioned reciprocally.

A gateway does two things, and they bundle because the same commercial and legal standing underwrites both:

- **Storefront** (§12) — renders signed catalogue objects into ordinary HTML over ordinary HTTPS, so a shopper with no TRACT client and no keypair can browse and buy. It offers stores a subdomain and accepts custom domains.
- **Settlement and escrow** (§9.6) — holds funds between order and delivery under its own payment-provider relationship, and rules on release or refund.

It is **not** the catalogue (that is the seller's feed), **not** the index (derived, and anyone may build one), **not** the courier, **not** the reputation authority (§10.4), and **not** a holder of anyone's identity keys — ever. Two TRACT-native parties never need a gateway to transact; it exists for the legacy web and for buyers who want recourse.

Gateways are **permissionless to enter and competing**: a seller may list through several at once, and moving between them costs a DNS change, because the store *is* the feed.

0.4.3 Honest departure from DMTAP's structural claim

DMTAP confines its one operator class to legacy SMTP egress, and that class **self-extinguishes** as adoption grows. TRACT's does not:

- The **storefront** function is permanent, because browsers are permanent. A shopper without a key-pair cannot verify a signature, so they trust the gateway rendered honestly. That is a real trust downgrade and it is disclosed as one (§12.6), mitigated but not removed by the fact that any node can re-render the same store and be compared byte-for-byte.
- The **escrow** function is permanent, because holding money for strangers is a licensed activity and physical custody cannot be made trustless (§9.6.4).

TRACT is therefore **structurally less pure than DMTAP**, and this document says so rather than discovering it later. What is preserved is that the class is *one*, entered permissionlessly, competed for, chosen per-order by both parties, replaceable without loss, and never in possession of identity keys.

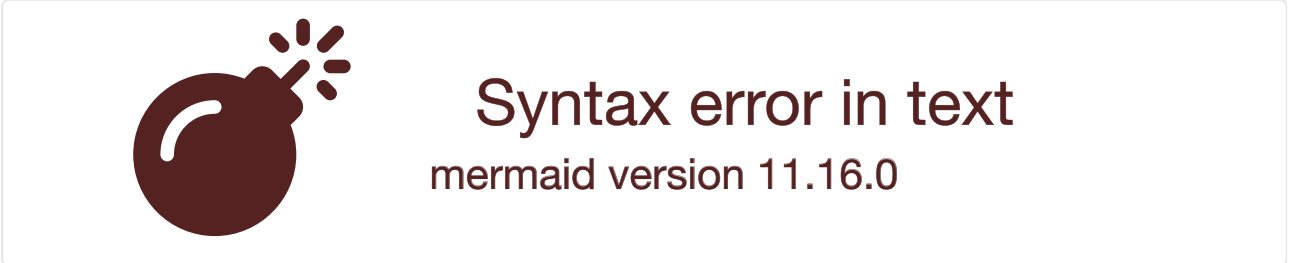
0.5 The two quadrants — what is public, what is sealed

The substrate's Feeds capability provides authenticity **without** confidentiality; its MOTE provides confidentiality **and** authenticity. TRACT splits commerce along exactly that seam, and the split is a hard rule, not a convention:

Public — signed, content-addressed, irrevocable	Sealed — encrypted, per-party, deletable
product records, offers, prices	orders, order lines
availability and stock signals	buyer name, address, contact
carrier and distributor rate cards	payment references and receipts
storefront render bundles	consignment routing detail
reviews (pseudonymous, §10.3)	dispute correspondence

> **Normative (§0.5.1). No personal data may enter the public quadrant.** Published objects are irrevocable and content-addressed; a right to erasure cannot be satisfied against them. An implementation MUST NOT publish, plaintext-address, or serve any object containing personal data. Orders are sealed, always. The residual case — reviews, which are public by nature and signed by a person — is bounded in §10.3 and disclosed in §14 as an honest limit.

0.6 The shape of a trade



The buyer's node is the orchestrator. There is no party that sees the whole cart, and no party whose absence prevents the trade — except a gateway, when the parties chose one.

0.7 Where state lives

State	Location	Notes
Catalogue, offers, rate cards, capacity	**Seller / courier / distributor node** (the edge)	published as signed feeds
Cart, wishlist, purchase history	**Buyer node** (the edge)	CRDT across the buyer's own devices; survives any store closing
Orders	**Both endpoints**, sealed	never in the middle, never public
Product records	**Content-addressed swarm**	globally deduplicated; any holder may serve
Indexes, search, rankings	**Anywhere**	derived, rebuildable, never authoritative
Funds in escrow	**A node in gateway mode**	the one irreducible operator function
Reachability (key → location)	**Substrate mesh / HTTPS**	signed, TTL'd, self-republished

Every row but the escrow row is either edge state or derived data. That is the invariant the rest of this document protects.

The liveness caveat, stated where the table is rather than late (§21.5). Durability at the edges covers *orders* — a sender's node retries, and an offline seller's orders wait. It does not cover *catalogues*. A seller whose node is offline is not slow; they are **invisible**. The one deployed system closest to this design measured a ~22-day median listing lifetime and whole catalogues disappearing when a merchant node departed. Any deployment therefore depends on third-party caching or pinning of public objects for sellers who are not always on — and unpaid replication is precisely what that system did not get. Whether pinning needs an incentive, and whether that incentive creates another operator, is open (§21.8).

0.8 Document map

§	Title	Covers
§1	Actors & identity	seller/buyer/courier/distributor/gateway identity, reachability
§2	Catalogue	product records, the product-identity ladder, offers, variants, indexes
§3	Availability	stock, time slots, capacity, made-to-order
§4	Fulfilment	ship, collect, digital grant, perform-at, perform-remote, access, return
§5	Consideration	fixed, tiered, recurring, metered, deposit, quote/RFQ; tax
§6	Cart	buyer-side CRDT cart, live availability, holds and reservations
§7	Order	the sealed order object and its state machine
§8	Delivery	rate cards, legs, consolidation, distributors, local route computation
§9	Settlement	the payment seam, rail classes, escrow, the gateway's money role
§10	Trust	purchase-attested reviews, local ranking, why no global score
§11	Jurisdiction	the four anchors, scope declarations, tax and consumer-law fields
§12	Gateway	storefront rendering, domains, origin isolation, honest trust limits
§13	Analytics	privacy-preserving measurement; what a merchant gains and loses
§14	Anti-abuse	listing spam, review manipulation, Sybil, and the honest limits

§	Title	Covers
§15	Conformance	profiles and the auditable fail-closed set
§16	Wire format	deterministic CBOR object definitions
§17	Errors & registries	the `ERR_TRACT_*` block and IANA-style registries
§18	State machines	offer, order, consignment, escrow
§19	Parameters	timeouts, limits, defaults
§20	References	normative and informative standards
§21	Grounding	what the evidence supports, and what it contradicts

0.9 Conventions & normative glossary

Requirement language. The key words MUST, MUST NOT, SHOULD, SHOULD NOT, MAY are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, in all capitals.

Glossary (normative). Defined once here, used with these meanings throughout.

- **product record** — a content-addressed public object describing *what a thing is*, not who sells it or for how much. Shared across sellers by construction (§2.2).
- **offer** — a signed announcement by **one** seller that it will supply a product record's subject on stated terms. An offer is always one seller's claim; a product record is nobody's (§2.4).
- **the four axes** — every offer declares **Item**, **Availability**, **Fulfilment**, and **Consideration** (§2.4.3). Together they express goods, services, rentals, subscriptions and bookings without category-specific machinery.
- **index** — derived, rebuildable aggregation over public feeds. **Never authoritative**; a disagreement with a feed resolves in favour of the feed (§2.6). The term *marketplace* is **not used** in this document for anything TRACT defines, precisely because an index is not one.
- **gateway** — **one sense only**: the operator-class role of §0.4.2 — storefront rendering and/or settlement and escrow. It never means an index, a courier, or a relay.
- **consignment** — physical goods in transit under someone else's custody: a courier leg or a distributor hold (§8.4).
- **leg** — one movement of goods between two places, priced by one rate card (§8.2).
- **rate card** — a signed public object from which a leg's price and transit estimate are **computed locally**, as opposed to quoted by a call to the carrier (§8.2).
- **place of supply** — the jurisdictional anchor derived from the **Fulfilment** axis, distinct from seller establishment, buyer residence, and delivery destination (§11.2). Getting these four confused is the most common commerce-tax error and this document keeps them separate by construction.
- **rail** — a settlement mechanism behind the payment seam, classified `CustodialReversible` or `NonCustodialFinal`. The class is part of the type because it changes the buyer's recourse (§9.2).
- **purchase attestation** — a signed proof that a review's author actually transacted, issued by the seller or by an escrow gateway (§10.2).
- **personal data** — as defined by the applicable regime (GDPR Art 4(1), POPIA §1, LGPD Art 5); for this document's purposes, any datum relating to an identified or identifiable natural person, **including a pseudonymous key that is linkable to one**.

1. Actors & identity

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

1.1 Scope

Who the parties are, how they are identified, and how they are reached. TRACT adopts substrate capability ① (Identity) unchanged and adds only the commerce-specific roles and disclosures.

1.2 What this section will specify

- The five participant roles — **seller, buyer, courier, distributor, gateway** — as roles of one identity type, never as separate account kinds. A single key may be all five.
- **Seller disclosure**: the fields an offer must carry to satisfy trader-traceability duties (legal name or trading name, contact route, establishment country, registration where held). This is the field set several consumer-protection regimes assume exists; §11 governs which apply.
- **Buyer disclosure tiers**: what the buyer's node attaches at browse, on grant, and at order.
- **Per-store pseudonymous subkeys**: derived so a buyer's activity is linkable *within* a seller (repeat-customer recognition, reviews) but not *across* sellers.
- **Reachability**: how a seller behind CGNAT is reached by key, and how an order is delivered to a node that is asleep or offline — the ladder, the mailbox, and wake.
- **The liveness asymmetry** (§21.5, §21.9). Durability at the edges covers *orders* and not *catalogues*, and this section has to make the distinction rather than let "reachable by key" imply both. A sleeping node still receives orders: the sender's node retains the retry queue and a content-free wake signal brings the recipient back. A sleeping node does **not** serve its catalogue, so an offline seller is not slow — they are invisible, and nobody else is obliged to serve their listings in their absence. The closest deployed relative of this design measured a ~22-day median listing lifetime and whole catalogues disappearing when a merchant node departed. Third-party caching or pinning of public objects therefore moves from a convenience to a practical requirement for any seller not always on, and unpaid replication is exactly what that system failed to attract. Whether pinning needs an incentive, and whether that incentive creates another operator, is open (§21.8).

1.3 Standards profiled

Ed25519 (RFC 8032), deterministic CBOR (RFC 8949 §4.2), COSE (RFC 9052), and the substrate's `DeviceCert`, key-transparency and key-name constructions. No new key type is introduced.

1.4 Open

- Whether a seller's legal-disclosure block belongs in the offer, the feed head, or both.
- Subkey derivation: per-store deterministic derivation is convenient but makes the link recomputable by anyone who learns the root key. A stored random mapping avoids that at the cost of recovery complexity.

2. Catalogue

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

2.1 Scope

Product records, offers, the product-identity ladder, variants, and the rules that keep indexes from becoming authorities.

2.2 The split, and why it is mechanical rather than conventional

A **product record** describes what a thing is; an **offer** is one seller's claim to supply it. Because the substrate content-addresses public blobs over plaintext, two sellers publishing the same record converge on the same address by construction, and the swarm stores it once. The global product view is therefore an emergent consequence of hashing, not a registry.

2.2a How strong the convergence claim actually is (§21.2)

Weaker than §2.2 sounds, and §21.9 obliges this section to say so rather than let a reader infer otherwise.

Convergence is trivially true for identical bytes and says nothing about the real case: two shops describing the same shoe. A July 2026 literature pass found **no deployed system achieving cross-publisher product identity without a licensed registry**, and the one candidate for permissionless crawl-derived resolution was refuted 0-3 under adversarial verification. The two models that exist in the field are the only two: a permissioned monopoly namespace (GS1 GTIN — licensed rather than sold, fee-bearing, issued through national member organisations) and a purely nominal string with no issuer or uniqueness guarantee (schema.org `productGroupID`). Nothing in between is deployed.

So the content address is a sound **mechanism** carrying an **unproven** claim. The canonicalisation rules of §2.3 — not the hashing — are the load-bearing part of this section, and §2.5 keeps near-duplicate resolution listed as open rather than implied-solved.

2.3 What this section will specify

- `ProductRecord` and `Offer` object shapes.
- **Canonicalisation**: the normalisation applied before addressing, since convergence is only useful to the extent independent publishers can actually produce identical bytes. This is the hard part of the section.
- The **identity ladder** — content address (floor, zero authority) → claimed external identifiers (advisory, unverified, squattable) → manufacturer-signed record (authority = the brand).
- **Variants**: product groups, varies-by axes, and per-variant records.
- **Bundles and kits**, including components published by other sellers.
- The **index rule**: derived, rebuildable, never authoritative; disagreement resolves toward the feed; no protocol mechanism exists by which an index can delist a seller from the network.

- **The gap between permission and practice** (§21.3, §21.9). "Any node may build an index" does not mean many will. A content-addressed substrate offers no global index, so discovery is the *first* function to re-centralize: whichever index becomes economically dominant becomes a de facto content-policy gatekeeper regardless of what this document permits. That is what happened to the closest deployed relative of this design, and the largest live decentralized-commerce network avoided it only by adopting a central approval-gating registry (§21.4). **No rule in this document prevents it.** Multiple competing indexers with verifiable completeness or censorship proofs is the candidate answer, and it has no deployed precedent (§21.8). This is the weakest load-bearing claim in the specification and is marked as such rather than defended.

2.4 Standards profiled

schema.org product vocabulary (`Product`, `Offer`, `ProductGroup`, `hasVariant`, `variesBy`) for the data model, so existing merchant feeds map in by translation. GS1 identifiers (GTIN, MPN) are supported as **claims only** — issuance is gated and fee-bearing, so a spec that depended on them would import a centralization point and a cost barrier.

2.5 Open

- Near-duplicate resolution. The address floor is exact-match; merging almost-identical records is an index-side heuristic, and heuristics differ between indexes. Whether the spec should recommend one, or deliberately leave it to differ, is undecided.
- Bootstrapping manufacturer signatures when most brands will not participate early.

3. Availability

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

3.1 Scope

The first of the four offer axes: when, and how much, is on offer.

3.2 Variants

`count` · `time-slots` · `capacity-per-interval` · `unlimited` · `made-to-order`

3.3 What this section will specify

- The availability object per variant, and how a buyer's node evaluates it locally.
- **Slot availability** as a profile of iCalendar, so a seller can publish from calendar software they already run rather than maintaining a second source of truth.
- **Lead time** for made-to-order, distinguished from stock-out.
- How availability signals are published without leaking commercially sensitive exact stock — a seller may publish a band ("in stock", "low") rather than a number.

3.4 Standards profiled

RFC 5545 `VAVAILABILITY` and `VFREEBUSY` for time-slot and capacity availability. Time zones per RFC 5545 / IANA tzdata.

3.5 Open

- Whether published availability bands need a normative vocabulary or stay seller-defined.

4. Fulfilment

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

4.1 Scope

The second axis: how the thing reaches the buyer — and, consequentially, **where the supply happens** for tax purposes.

4.2 Variants

`ship` · `collect` · `digital-grant` · `perform-at-place` · `perform-remote` · `access-grant` · `return-required`

4.3 What this section will specify

- The fulfilment object per variant, including the place a service is performed and the return terms for a rental.
- The **place-of-supply derivation table** — the mapping from fulfilment variant to jurisdictional anchor (§11.2). Ship → destination; perform-at-place → the venue; perform-remote and digital-grant → buyer residence; return-required → collection point.
- **Handover**: what constitutes delivery for each variant, and what evidence is signed by whom.
- Multi-variant offers (collect *or* ship) and how the buyer's choice binds the anchors.

4.4 Standards profiled

Incoterms 2020 for the risk/cost transfer point on shipped goods. ISO 3166 for places.

4.5 Open

- Whether digital-grant needs a licence-terms sub-object or should defer entirely to §5.

5. Consideration

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

5.1 Scope

The third axis: what is paid, on what schedule, and how tax attaches to it.

5.2 Variants

`fixed` · `tiered/volume` · `recurring` · `metered` · `deposit+balance` · `quote-required (RFQ)`

5.3 What this section will specify

- Money representation: **minor units and currency code, never a float**.
- Per-variant pricing objects, including recurrence rules and metering dimensions.
- **Quote/RFQ**: a signed request and a signed counter-offer, which is also the B2B contract-pricing mechanism — a per-buyer price is an offer addressed to one key.
- **Tax presentation**: inclusive vs exclusive display, and which anchor determines the rate.
- Currency conversion as a **presentation** concern, with the settlement currency binding.

5.4 Standards profiled

ISO 4217 currency codes. RFC 5545 recurrence rules for subscription periods, reusing §3's machinery rather than inventing a second schedule format.

5.5 Open

- Whether tax *rates* belong in the protocol at all. Current position: no — rates change by jurisdiction and by week; the offer carries the treatment category and the anchors, and rate lookup is an implementation concern. Encoding rates would guarantee the spec ships stale law.

6. Cart

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

6.1 Scope

Buyer-side cart state, live availability, and reservation semantics without a central lock.

6.2 What this section will specify

- The cart as **buyer-held CRDT state**, synced across the buyer's own devices via substrate capability ③. No seller and no gateway holds it.
- Live re-evaluation: subscribing to availability feeds for items already in the cart.
- **Reservation and hold** semantics, including what a seller may promise and for how long.
- **Bounded-counter inventory** across a seller's own replicas: stock partitioned into per-replica quotas, sold freely within quota, quota transferred between replicas on demand. The invariant is that total sales never exceed total stock, without a coordinator.

6.2a What the bounded counter actually costs (§21.10)

The mechanism is real and shipped: the escrow/demarcation family (O'Neil 1986 → Barbará-Millá & Garcia-Molina 1994 → Balegas et al., SRDS 2015), in production as AntidoteDB's `antidote_crdt_counter_b`. Safety holds under arbitrary message loss, delay, reorder and partition, because the locally computed right count is conservative — it can under-count but never over-count. For contrast, a plain read-check-decrement counter measurably *does* oversell: about 200 excess decrements at ~200 concurrent clients in the published experiment.

What §21.10 surfaced is that **four operator-shaped roles hide inside it**, and this section has to accept or replace each rather than let "no coordinator" read as "no coordination anywhere":

1. **An intra-replica serialization point is mandatory** — a single-writer node, a compare-and-swap, or consensus. A replica therefore cannot itself be a set of mutually uncoordinated nodes. The coordination is not removed; it is pushed down one level and made local.
2. **The failure model is crash-recovery with durable state, not Byzantine.** This is the hard boundary. A lying or state-losing replica simply oversells, and the literature covers **a seller's own replicas, which trust each other** — not stock held across mutually distrusting parties. Bounded counters protect a seller from their own concurrency; they do **not** protect a buyer from a dishonest seller, and conflating the two would be a safety claim nothing supports.
3. **Reclaiming a dead replica's rights is a fallible decision.** If the failure detector is wrong, rights are double-issued and the invariant breaks. Deciding a peer is permanently gone is exactly the judgement a coordinator makes, so it needs a stated failure mode rather than a background sweep.
4. **At three or more replicas, transfer names a recipient.** It is no longer anonymous, which reintroduces an allocation policy — and the source papers suggest centralising it. This section has to say who chooses.

6.3 Prior art

Escrow-based / bounded CRDT counters from the distributed-systems literature. Saga and compensating-transaction patterns for the multi-party case.

6.4 Honest limits this section must state

- A partitioned replica holding unused quota **strands** that stock until it rejoins.
- There is **no cross-seller atomicity**. A multi-seller cart is a set of independent orders; one seller declining does not roll back another's acceptance. Checkout is compensating actions, never a dis-

tributed transaction — that would need a coordinator with authority over sovereign parties. Interfaces must show per-seller status rather than a single “order placed”.

6.5 Open

- Quota rebalancing policy: how aggressively replicas should transfer, and whether the spec should mandate one or leave it to implementations.

7. Order

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

7.1 Scope

The sealed order object, its lifecycle, and the evidence each transition produces.

7.2 What this section will specify

- `Order` as a **sealed message**, one per seller, carrying only that seller’s lines. It is never published; it lives at the two endpoints and is deletable there.
- The state machine: draft → placed → accepted / declined / countered → fulfilling → delivered → closed, with cancellation and return paths, and the timeout at each edge.
- **Signed transitions**: which party signs what, so that acceptance, dispatch, custody handoff and receipt are each provable after the fact.
- Order amendment, partial fulfilment, and partial refund.
- **Returns and exchanges**, including who pays return carriage under which fulfilment variant.

7.3 Standards profiled

The substrate’s sealed message object for transport. Where an order must be exported to a legacy counterparty (an ERP, a 3PL), the mapping targets are UN/CEFACT and EDIFACT order/despatch/invoice messages; that mapping is informative, not required.

7.4 Open

- Whether order amendment is a new sealed object superseding the prior, or an operation log. The supersede model matches the rest of the design; the log model is easier to audit.

8. Delivery

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

8.1 Scope

Rate cards, legs, consignments, consolidation, and local route computation.

8.2 What this section will specify

- `RateCard` as a signed public object: zone table, weight brackets, dimensional divisor, surcharges, transit estimates, served countries, excluded categories.
- The **local computation** rule: a buyer's node computes leg prices from published rate cards rather than calling a quote API — so there is no rate limit, no API key, and no third party learning what is being bought.
- `Leg` and `Consignment` objects, and signed **custody handoff** attestations.
- **Consolidation**: the candidate routings (direct, hub-near-buyer, hub-near-sellers) and the comparison `Σ legs + storage × wait + handling`. The candidate set is small enough to evaluate exhaustively; no optimiser is required or specified.
- Distributor capacity objects.

8.3 Standards profiled

Incoterms 2020 for transfer of risk. GS1 SSCC for logistic-unit identification where the parties already use it. ISO 3166 for served territories. UPU conventions for cross-border postal legs.

8.4 Honest limits this section must state

- Some carriers restrict redistribution of negotiated rates. Published list rates and a seller's own rates they choose to publish are fine; republishing confidential rates is the publisher's compliance problem. Offers may declare that a live quote is required instead.
- Transit estimates are estimates. `wait_days` for consolidation is the least reliable input in the whole computation and must be presented as an estimate.
- Custody attestations prove **transfer**, not **recoverability**. Loss and damage are §9's problem, or nobody's.

8.5 Open

- Whether a peer courier needs a distinct rate-card shape (flat per-km) or should express that within the zone-table model.

9. Settlement

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

9.1 Scope

The payment seam, rail classes, escrow, and the gateway's money role.

9.2 The seam names no provider

TRACT specifies where money crosses the boundary and what must be verified. It specifies no rail, currency, token or ledger. Naming a provider in a protocol exports that provider's jurisdiction and licensing to every implementor.

9.3 Rail class is part of the type

`CustodialReversible` (chargebacks exist; the card network is already the adjudicator) versus `NonCustodialFinal` (nobody custodies, nothing reverses). An implementation must not substitute one class for the other without an explicit party decision, because the buyer's recourse differs.

9.4 What this section will specify

- The payment-attestation object: payer, payee, order, amount, rail class, external settlement reference. **The protocol carries attestations, never funds.**
- Escrow lifecycle: fund → hold → release / refund / split, each a signed object.
- `EscrowScope`: buyer countries, seller countries, supply countries, currencies, rail classes, value ceiling, excluded categories, claimed authorisations.
- **Fail-closed scope intersection** at checkout, and the requirement that an empty intersection is disclosed rather than silently downgraded.

9.5 Escrow is the operator class

It requires legal standing, a payment-provider relationship, a float, and licensing — none of it derivable from a keypair. What bounds it: permissionless entry, competition, per-order choice by both parties, no access to identity keys, and **every ruling published as a signed object**, so an operator that rules unfairly accumulates a permanent verifiable record.

9.5a The measured cost of optionality (§21.6)

§9.5 keeps escrow per-order and optional, and §9.6 gives the reason: mandatory escrow would exclude regions no licensed operator serves. That reasoning is sound and the consequence is still real.

OpenBazaar's escrow was also opt-in, and **bad actors simply declined it** — the protection went unused precisely where it mattered most. Both cannot be true for free, and this specification pays on the second. What follows from that, rather than pretending it away:

- an unescrowed trade is an explicit, disclosed outcome shown to both parties before they commit, never a silent default;
- interfaces render the absence of escrow as a fact about the trade, not as a missing field;
- a seller declining every escrow provider a buyer trusts is itself information the buyer should be given, since it is the observable form of the failure above.

9.6 Honest limits this section must state

- Physical custody cannot be made trustless.
- Non-custodial programmatic escrow (multi-sig, hashlock+timelock) removes the custodian but **deadlocks on genuine disputes** — the exact case it was wanted for.

- Unescrowed trade must remain possible, or scope mismatches would exclude underserved regions.

9.7 Open

- Whether partial release (split rulings) needs protocol representation or is an operator concern.

10. Trust

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

10.1 Scope

Reviews, purchase attestation, and ranking — without creating an authority.

10.2 What this section will specify

- `Review` as a signed public object attached to a product address, seller, distributor or courier.
- **Purchase attestation**: a proof, issued by the seller or an escrow operator at completion, that the author actually transacted. Verifiable by anyone, unlike a platform's "verified purchase" badge, and it makes ballot-stuffing cost a real transaction.
- Per-seller pseudonymous authorship (§1.2).
- Seller responses, and supersede-based retraction.

10.3 The prohibition

No network-wide published score. Computing one requires a party that aggregates and ranks, which is the authority the design removes. Ranking is derived data; indexes will differ, and that is the intended outcome, not a defect.

10.3a What attestation does not fix (§21.6)

§21.9 obliges this section to state the measured outcome rather than the hypothetical one.

OpenBazaar's reputation failure was self-published, unweighted reviews with no banning authority, and **one vendor faked 60% of measured sales value**. A purchase attestation is strictly stronger than what OpenBazaar had — it binds a review to a real transaction, so ballot-stuffing costs actual trades. Two of the failure conditions nonetheless survive unchanged:

1. **Self-dealing produces genuine attestations.** A seller transacting with itself generates real proofs. Attestation raises the cost and leaves a public trail; it does not establish that the counterparty was independent, and nothing in this document does.
2. **Escrow-issued attestations are scarcest where they would matter most**, because escrow is opt-in and declined by exactly the actors it constrains (§9.5a).

The achievable Sybil-cost floor on a signed-feed substrate is an **open question** (§21.8), not a solved one. An index's weighting policy is where that judgement lives, and different indexes reaching differ-

ent answers is the design rather than a defect.

10.4 Honest limits this section must state

- Rankings disagree between indexes; buyers lose the convenience of one authoritative number.
- Attestation raises manipulation cost but does not eliminate it — a seller can transact with itself, expensively and visibly.
- Whitewashing is bounded, not solved: a new key has no history, and discounting new keys is the only available defence.
- **Erasure**: a review is public, irrevocable, and signed by a person. Retraction is a superseding tombstone honoured by conformant clients and gateways; the residual — bytes persist if any independent holder keeps them — must be disclosed to the author before publishing.

10.5 Prior art

Sybil, whitewashing and ballot-stuffing literature on decentralized reputation; web-of-trust and locally-measured reputation as the alternatives to a global score.

11. Jurisdiction

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

11.1 Scope

Making the legally responsible party explicit, and getting tax anchors right by construction.

11.2 The four anchors

The most common commerce-tax error is conflating party location with where a supply happens. These are four distinct fields, derived from four different places:

Anchor	Derived from	Governs
seller establishment	seller identity	licensing, seller-side registration
buyer residence	buyer disclosure at order	consumer-protection rights (generally non-waivable)
place of supply	**the Fulfilment axis (§4)**	VAT/GST, especially services and events
delivery destination	the shipping leg (§8)	customs, duty, product-safety regimes

The forcing example: an event held in one country, sold by a seller in another, to a buyer in a third. Admission to events is generally taxed where the event physically takes place, so only the Fulfilment object knows the answer.

11.3 What this section will specify

- **Responsibility follows the money**: every order names seller of record, facilitator (the gateway, if it settled), importer of record, in-region responsible person, and escrow/rail.
- Geo-availability on offers, and fail-closed construction when a required in-region role is absent.
- Tax treatment categories (not rates — see §5.5).

11.4 Regimes this section must accommodate

South Africa (electronic-transaction disclosure and cooling-off, consumer protection, POPIA, VAT, payment-side KYC); the EU (GDPR, platform trader traceability, consumer rights, in-region responsible person for product safety, VAT one-stop schemes, platform reporting); other African markets (national data-protection acts, local VAT registration, regional trade frameworks); New Zealand (privacy, fair trading, consumer guarantees, GST on low-value imports); the Americas (US economic nexus and marketplace-facilitator rules, seller-traceability legislation, Canadian and Brazilian privacy law).

The protocol guarantees the **facts** a regulator asks for are present, signed and attributable. It does not make any deployment compliant, and must not be read as legal advice.

11.5 The erasure conflict, resolved structurally

Published objects are irrevocable; erasure rights cannot be satisfied against them. Therefore **no personal data enters the public quadrant** (§0.5.1). Orders and everything identifying a person are sealed and deletable at the edges. Reviews are the single bounded exception (§10.4).

12. Gateway

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

12.1 Scope

The storefront role: rendering signed objects for browsers, domains, and the trust boundary.

12.2 What this section will specify

- The rendering contract: what a gateway must verify before serving, and what it publishes about what it served.
- **Render bundles**: seller-supplied presentation, sandboxed.
- **Subdomain** allocation (single-label, one wildcard certificate) and **custom domain** binding.
- Escrow co-location: a gateway that also settles is the facilitator of §11.3.

12.3 Origin isolation (will be normative)

Merchant render bundles are untrusted code. **Every store must get its own origin.** A gateway serving multiple stores from one origin lets one bundle read another store's cart and session; it is non-con-

formant, not merely ill-advised.

And this section has to say how origins are compared, which it previously did not. Implementing the rule surfaced the gap: a naive string comparison reports `alice.example` and `Alice.example` as distinct origins, while DNS and every browser treat them as the same one — same storage partition, same cart, same session. A gateway that allocated both would believe it had isolated two stores and would in fact have handed one merchant's bundle read access to the other's data, while passing its own conformance check.

So the comparison is normalised before it is made: ASCII-lowercased, with any trailing root label dot removed. Internationalised names need an IDNA pass before that point, and a gateway accepting them without one has the same hole in a less obvious form. A rule that says *what* must be true and not *how it is tested* is a rule an implementation can satisfy on paper and violate in practice.

12.4 Process isolation (will be normative)

A gateway terminates untrusted connections and renders untrusted bundles. It must run as a separate process with no access to identity keys or the object store. "One binary, several roles" never means one address space.

12.5 The honest limit

DMTAP's gateway self-extinguishes as adoption grows. **TRACT's does not**, because browsers are permanent and a shopper without a keypair cannot verify a signature. The mitigation is **detectability, not prevention**: any node can re-render the same store from the same signed objects and be compared byte-for-byte. A native client needs no gateway; two native parties never need one to transact.

12.6 Open

- Whether the spec should define a canonical render so byte-for-byte comparison is meaningful, or only require that the underlying objects be exposed for independent verification.

13. Analytics

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

13.1 Scope

Measurement for sellers that does not require per-visitor surveillance.

13.2 Posture

Tiered and buyer-granted, with an aggregate floor: anonymous browse by default; scoped signed disclosure the buyer's node chooses to attach; full detail at order, which the seller needs to fulfil anyway; aggregate telemetry carrying no per-visitor record.

13.3 What this section will specify

- The grant object: what a buyer's node may disclose, to whom, for how long, revocably.
- The aggregate telemetry object and its privacy parameters.
- Anonymous rate-limiting credentials for bot and abuse signals without identity.

13.4 Standards profiled

Privacy-preserving aggregation and attribution work from the browser vendors; anonymous credential schemes for un-attributed rate limiting; Oblivious HTTP (RFC 9458) where IP-level unlinkability is wanted.

13.5 What a seller loses, stated plainly

No cross-site retargeting; campaign-level rather than person-level attribution; no individual session replay; coarser fraud signals on non-reversible rails. Aggregation needs volume, so small sellers get noisier numbers. A section claiming parity with hosted analytics would be false.

14. Anti-abuse

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

14.1 Scope

Listing spam, fake stores, review manipulation, Sybil identities, and resource exhaustion.

14.2 The structural position

A seller can flood only **their own** feed, which only their own followers and holders pay for and may stop serving at will. There is no shared feed to spam and no fan-out amplification. This is inherited from the substrate's public-object model and is why catalogue spam is a weaker threat here than on a shared marketplace.

14.3 What this section will specify

- Holder admission policy: object-size ceilings, per-publisher storage quota, feed-append rate.
- Index admission policy, and the requirement that refusal to serve or index is a **policy decision**, never a protocol-level takedown.
- Cold-contact economics for unsolicited sealed messages (an order from a stranger is exactly the case the substrate's anti-abuse tiers were designed for).
- Review manipulation: what purchase attestation does and does not prevent.

14.4 Honest limits

- Attestation makes fake reviews expensive, not impossible.

- A new identity has no history; discounting new keys is the only defence, and it penalises legitimate newcomers. There is no resolution to this that does not create an authority.

15. Conformance

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

15.1 Scope

Profiles, the auditable fail-closed set, and the conformance vectors.

15.2 Planned profiles

A node need not implement everything. Provisional profiles: **catalogue-only** (publish and serve offers), **transacting** (adds sealed orders), **routing** (adds delivery computation), **gateway** (adds storefront and/or settlement).

15.3 The fail-closed set

Every security-relevant failure is either refused or surfaced as an explicit choice, never a silent degradation. The table will index every must whose violation must fail closed, with the owning clause authoritative — in particular scope-intersection failure (§9.4), rail-class substitution (§9.3), origin isolation (§12.3), and the public-quadrant personal-data prohibition (§0.5.1).

15.4 Vectors

Frozen test vectors under `conformance/`, so an independent implementation proves byte-identity rather than plausible behaviour.

16. Wire format

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.
>
> The grammar below is **proposed, not frozen**. It is written out rather than deferred because a data model argued in prose hides its gaps, and because §15's conformance vectors cannot exist until these shapes settle (`conformance/README.md`). Every object here is implemented and exercised end to end in the reference implementation, which is evidence the shapes compose — and is *not* evidence that they are right, since an implementation and a grammar derived from each other only prove they agree with one another (§21.7, and the DMTAP case in `conformance/README.md`).

16.1 Scope

The byte-level definition of every TRACT object: what is encoded, in what order, with which keys, and which of those keys a decoder may not tolerate.

16.2 Conventions inherited, not reinvented

Deterministic CBOR (RFC 8949 §4.2). Integer-keyed maps, keys assigned per object type from 1, keys ≥ 64 reserved for extension. Signed objects reject unknown keys fail-closed; unsigned objects may ignore unknown keys ≥ 64. Domain-separation tags on every signing preimage. Content addresses carry a multihash-style agility prefix.

TRACT introduces no new hash construction, no new signature framing, and no new address scheme. All four come from the DMTAP substrate. A construction invented in this section is a defect.

16.3 Shared primitives

```
content-address = bytes         ; multihash-style prefix ‖ digest (substrate §18.1.5)
identity-key    = bytes         ; the public half of an IK; also a courier, distributor
or gateway
ts              = int           ; milliseconds since the Unix epoch
country         = tstr .size 2  ; ISO 3166-1 alpha-2
currency        = tstr .size 3  ; ISO 4217

money = {
  1 => int,        ; minor_units — NEVER a floating-point value (§16.7)
  2 => currency,
}
```

16.4 The structural rule — two type families, not one with a flag

Public and sealed objects are **separate type families**. A public object is structurally incapable of carrying a name, address or contact detail (§0.5.1): the prohibition is enforced by the grammar, not by reviewer discipline, because published objects are irrevocable and an erasure right cannot be satisfied against them after the fact.

Two consequences the grammar has to carry, rather than leaving to convention:

- **A locality, never an address.** Public objects that reference a place (`PlaceRef`, `CapacityRecord`) carry a country and a coarse locality only. There is no street-address production in the public family at all, so “just add a line” is a grammar change a reviewer sees rather than a field a contributor adds.
- **No boolean discriminator.** A decoder is always in exactly one mode — expecting a public object or a sealed one — and the two are made non-confusable at the *content-address* level by distinct domain-separation tags, in the same way the substrate separates its own sealed and public manifests. There is no `sealed: true` field an attacker could omit, because a flag can be dropped and a domain-separation tag cannot.

16.5 Public objects

16.5.1 `ProductRecord` — what a thing is

Belongs to nobody. Two publishers whose canonicalised records encode identically converge on one content address (§2.2), with the caveat §2.2a records about how weak that is in the real case.

```
ProductRecord = {
  1 => tstr,                    ; name        canonicalised (§2.3)
  2 => tstr,                    ; description canonicalised; may be empty
  3 => [* Attribute],           ; attributes  sorted, deduplicated, keys casefolded
  4 => [* IdentityRung],        ; identity    weakest first
  ? 5 => content-address,       ; group       the ProductGroup this varies within
  ? 6 => [* content-address],   ; components  bundle members; may reference other sell-
ers' records
}

Attribute = { 1 => tstr, 2 => tstr }    ; key (casefolded), value

IdentityRung = ContentAddressRung / ClaimedExternalRung / ManufacturerSignedRung
ContentAddressRung     = { 0 => content-address }
ClaimedExternalRung    = { 1 => tstr, 2 => tstr }   ; scheme ("gtin", "mpn"), value —
UNVERIFIED
ManufacturerSignedRung = { 2 => identity-key }      ; the brand's own key
```

The middle rung is a **claim and nothing more**: anyone can assert any GTIN, so an index treats it as an advisory join key and never as authority (§2.3). Squatting is expected rather than prevented.

16.5.2 `Offer` — one seller's claim to supply

```
Offer = {
  1 => Item,
  2 => Availability,
  3 => Fulfilment,
  4 => Consideration,
  5 => [* country],            ; sell_to — territories this offer may be accepted from
  6 => ts,                     ; published
}

Item = { 0 => content-address }                              ; product
     / { 1 => content-address, 2 => content-address }        ; variant-of-group: group,
variant
     / { 3 => content-address }                              ; service
     / { 4 => content-address }                              ; right / licence
     / { 5 => content-address }                              ; capacity

Availability = { 0 => StockSignal }
             / { 1 => tstr, 2 => uint }        ; time-slots: RFC 5545 payload, slot
minutes
             / { 3 => uint, 4 => tstr }        ; capacity-per-interval: capacity, RFC
5545 recurrence
             / { 5 => null }                   ; unlimited
             / { 6 => uint }                   ; made-to-order: lead days

StockSignal = { 0 => uint } / { 1 => null } / { 2 => null } / { 3 => null }
            ; exact(n) / in-stock / low / out-of-stock — a band is publishable instead
of a number,
            ; because exact stock is commercially sensitive and browsing does not re-
quire it

Fulfilment = { 0 => [* country] }              ; ship: destinations served
           / { 1 => PlaceRef }                 ; collect
           / { 2 => null }                     ; digital-grant
           / { 3 => PlaceRef }                 ; perform-at-place — the venue
           / { 4 => null }                     ; perform-remote
           / { 5 => PlaceRef / null }          ; access-grant, physical or not
           / { 6 => PlaceRef, 7 => uint }      ; return-required: place, term days

PlaceRef = { 1 => country, 2 => tstr }         ; country, LOCALITY — never a street ad-
dress (§16.4)

Consideration = { 0 => money }                           ; fixed
              / { 1 => [+ PriceTier] }                   ; tiered / volume
              / { 2 => money, 3 => tstr }                ; recurring: amount, RFC 5545
RRULE
              / { 4 => tstr, 5 => money }                ; metered: dimension, unit
price
              / { 6 => money, 7 => money }               ; deposit + balance
              / { 8 => null }                            ; quote-required (RFQ; also B2B
contract pricing)

PriceTier = { 1 => uint, 2 => money }          ; min_qty, unit_price
```

The load-bearing detail. `Fulfilment` is not merely logistics: it is the only object that knows where a supply happens, and §11.2 derives the tax anchor from it. Ship → destination; collect / perform-at-

place / return-required → the stated place; perform-remote and digital-grant → buyer residence; access-grant → whichever, depending on whether it names a place. An implementation that resolves place of supply from the parties' countries instead will be plausibly and consistently wrong about every event held abroad.

16.5.3 `RateCard` and `CapacityRecord` — published, so routing is computed not quoted

```
RateCard = {
  1 => identity-key,            ; carrier — a multinational and a neighbour with a van,
identically typed
  2 => [* country],             ; serves
  3 => [* Zone],
  4 => uint,                    ; dim_divisor — billable = max(actual, L*W*H / divisor)
  5 => uint,                    ; surcharge_pct
  6 => [* tstr],                ; excluded_categories
  7 => ts,                      ; published — cards drift; a stale one is not replayed
silently
}

Zone          = { 1 => uint, 2 => [+ WeightBracket], 3 => uint }  ; id, brackets, tran-
sit_days
WeightBracket = { 1 => uint, 2 => money }                          ; max_grams, price

CapacityRecord = {                                ; a distributor's published consoli-
dation offer
  1 => identity-key,            ; distributor
  2 => country,
  3 => tstr,                    ; locality — coarse (§16.4)
  4 => money,                   ; storage_per_day
  5 => money,                   ; handling_fee
  6 => uint,                    ; slots
  7 => [* tstr],                ; excluded_categories
}
```

A rate card is a **claim by its publisher**, and a transit figure inside it is an estimate. §8.4 records that some carriers restrict redistribution of negotiated rates, which is why an offer may instead declare that a live quote is required.

16.5.4 `EscrowScope` — what an operator can lawfully serve

```
EscrowScope = {
  1 => identity-key,            ; operator
  2 => [* country],             ; buyer_countries
  3 => [* country],             ; seller_countries
  4 => [* country],             ; supply_countries — checked against place of supply,
not the parties
  5 => [* currency],
  6 => [* RailClass],
  7 => money,                   ; max_order_value — usually a KYC threshold
  8 => [* tstr],                ; excluded_categories
  9 => [* tstr],                ; authorities claimed — prose, because regulators share
no schema
}

RailClass = 0 / 1              ; 0 = custodial-reversible, 1 = non-custodial-final
```

`supply_countries` is a separate field from the party countries deliberately: two trades with identical buyers, sellers, currency and rail can differ only in where the supply happens, and that difference alone can put one of them outside an operator's licence (§9.4).

16.5.5 `Review` and `PurchaseAttestation` — the bounded exception

```
Review = {
  1 => Subject,
  2 => identity-key,            ; author — a PER-SUBJECT pseudonymous subkey, never the
root IK
  3 => uint,                    ; score, 0..5
  4 => tstr,                    ; body
  ? 5 => PurchaseAttestation,
  6 => ts,
}

Subject = { 0 => content-address }   ; product
        / { 1 => identity-key }      ; seller
        / { 2 => identity-key }      ; distributor
        / { 3 => identity-key }      ; courier

PurchaseAttestation = {
  1 => 0 / 1,                   ; attestor: 0 = seller, 1 = escrow operator
  2 => identity-key,            ; issuer
  3 => content-address,         ; order — the SEALED order's address only; never its
contents
  4 => ts,
}
```

A review is the one public object signed by a natural person, and §10.4 bounds it rather than exempting it. Two things the grammar carries: the author is a per-subject subkey so reviews are not trivially linkable across sellers, and the attestation references a sealed order **by address only** — nothing about what was bought crosses into the public quadrant.

`body` remains free text a person could type an address into. The grammar cannot prevent that; §10.4 and the client requirements have to, and pretending otherwise would be exactly the kind of overclaim

§16.4 exists to avoid.

16.6 Sealed objects

Never published. Held at the two endpoints, where deletion is meaningful.

```
Order = {
  1 => identity-key,            ; buyer
  2 => identity-key,            ; seller
  3 => [+ OrderLine],           ; THIS seller's lines only — never the whole cart
  4 => money,                   ; total
  5 => OrderState,
  6 => ts,                      ; placed
  ; buyer name, delivery address and contact details are carried here, in the sealed
family,
  ; and have no production in the public family at all (§16.4)
}

OrderLine  = { 1 => content-address, 2 => identity-key, 3 => uint }   ; offer, seller,
quantity
OrderState = 0 / 1 / 2 / 3 / 4 / 5 / 6 / 7 / 8
           ; draft / placed / accepted / declined / countered / fulfilling / delivered
/
           ; closed / cancelled  (§18)

PaymentAttestation = {
  1 => identity-key,            ; payer
  2 => identity-key,            ; payee
  3 => content-address,         ; order
  4 => money,
  5 => RailClass,               ; determines the buyer's recourse — never flattened to
a boolean
  6 => tstr,                    ; external settlement reference, opaque
  7 => ts,
}
```

One order per seller is a grammar-level property, not a client convention. `Order` names a single seller and carries only that seller's lines, so a cross-seller order is not expressible. The whole-cart view exists on the buyer's device and nowhere else (§6.1).

`PaymentAttestation` carries a *reference*, never funds and never card data. The protocol conveys that a payment happened; it does not move money (§9.2).

16.7 Encoding rules that exist because of a specific failure

- **Money is minor units and a currency code, never a float.** A rounding error in a signed object cannot be corrected after the fact — the signature covers the wrong number.
- **Arithmetic across currencies is refused, never coerced.** A silently converted total is a wrong total that looks right, and it gets carried into an order (§5.3).
- **A decoder that finds a field it does not recognise in a signed object rejects it**, rather than ignoring it. The alternative lets a signer and a verifier disagree about what was signed.
- **`ts` is milliseconds since the Unix epoch**, subject to the substrate's clock-skew tolerance. A timestamp is for display and ordering; where order must be authoritative it comes from a feed's se-

quence, never from a clock.

16.8 Open

- **Whether `Offer` carries its own signature or inherits authenticity from its position in a signed feed.** The substrate's feed head transitively commits to every entry, which makes a per-object signature redundant — and makes an offer quoted out of its feed unverifiable. The trade is real and undecided.
- **Whether `sell_to` being empty means "unrestricted" or "malformed."** Unrestricted is convenient and is almost never what a seller means, given §11's in-region representative requirements. Leaning malformed.
- **Extension-key policy for the axis unions.** Each axis is a small closed set today. What a decoder does with an axis variant it has never seen — refuse, or preserve and refuse only on *acting* — is §17.4's tolerant-to-store / strict-to-act rule, and it needs stating here in grammar terms rather than only in prose.

17. Errors & registries

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

17.1 Scope

The `ERR_TRACT_*` code block, the responder-action vocabulary, and the extension registries.

17.2 Conventions

Codes are allocated in a subsystem block, each with: name, operation, meaning, retryability, and a responder action drawn from a closed vocabulary (fail-closed-block, drop-silent, rotate-retry, deny-policy, halt-alert).

17.3 Registries planned

Item kinds · availability variants · fulfilment variants · consideration variants · rail classes · tax treatment categories · excluded-category vocabulary · external-identifier schemes.

17.4 The rule that makes registries safe here

Every registry above is **extensible**, and an unrecognised value must not be fatal to a generic index — it is preserved and surfaced as unknown. But a client that *renders or transacts* against a variant it does not implement must refuse rather than guess. Tolerant to store, strict to act.

18. State machines

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

18.1 Scope

The state machines, collected in one place so they can be checked against §7's prose rather than inferred from it — and so that the question this section exists to force gets asked of every edge: **what happens when the counterparty never answers?**

That question is the whole point. A protocol between sovereign parties has no supervisor to notice a stalled trade, so a transition without a defined expiry is not an edge case, it is where money and goods sit indefinitely with nobody empowered to move them. Every table below therefore carries a timeout column, and "none" appears only where waiting forever is genuinely correct.

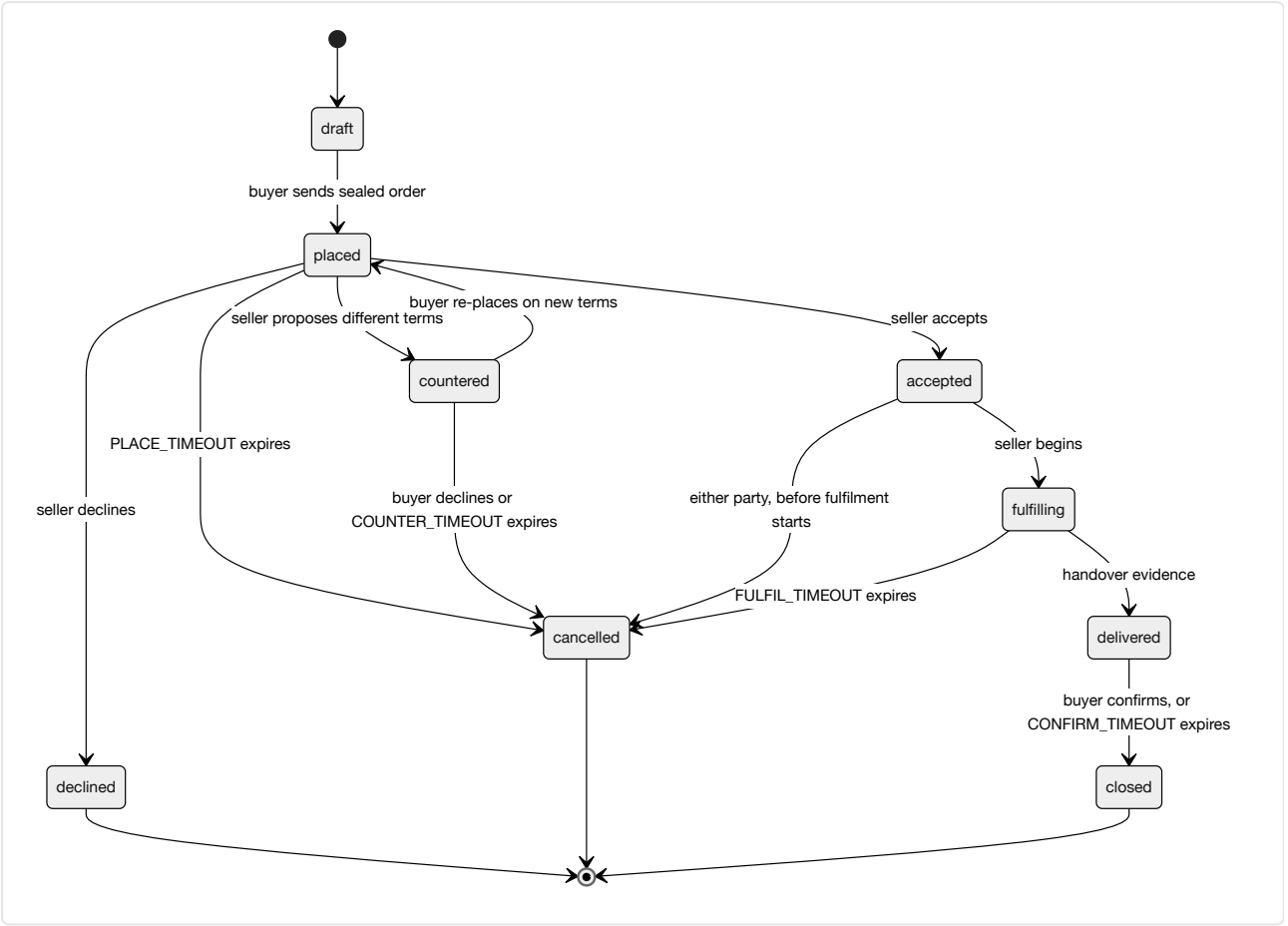
18.2 Offer

The simplest machine, and the only one with no counterparty: an offer is one seller's unilateral publication.

From	To	Trigger	Signed by	Timeout
—	`published`	the seller publishes	seller	none
`published`	`superseded`	a later offer replaces it	seller	none
`published`	`withdrawn`	the seller stops offering	seller	none

Withdrawal is not deletion. A published offer is content-addressed and irrevocable (§0.5), so `withdrawn` is a successor object stating the offer no longer stands, not an erasure of the original. A buyer holding a cached offer learns it is withdrawn by fetching the seller's feed; until they do, they may present terms the seller has stopped honouring, which is why acceptance is the seller's (§18.3) and not automatic.

18.3 Order



From	To	Trigger	Signed by	Timeout → behaviour
`draft`	`placed`	buyer sends	buyer	none — a draft is local
`placed`	`accepted` / `declined` / `countered`	seller responds	seller	`PLACE_TIMEOUT` → `cancelled`. **A silent seller cancels; it never accepts.**
`countered`	`placed` / `cancelled`	buyer responds	buyer	`COUNTER_TIMEOUT` → `cancelled`
`accepted`	`fulfilling`	seller begins	seller	none
`fulfilling`	`delivered`	handover evidence (§18.4)	carrier or seller	`FULFIL_TIMEOUT` → `cancelled`, and escrow refunds (§18.5)
`delivered`	`closed`	buyer confirms	buyer	`CONFIRM_TIMEOUT` → `closed`. **A silent buyer closes; escrow releases.**

The two asymmetric defaults are deliberate and pull in opposite directions, which is the honest part:

- **Silence before acceptance cancels.** A seller who never answers must not be bound, and a buyer must not have funds committed against an order nobody acknowledged.
- **Silence after delivery closes.** A buyer who received goods and then stops responding must not be able to strand the seller's money forever by doing nothing. This is the one place the protocol favours the seller, and it does so because the alternative — funds held until a buyer affirmatively acts — makes non-response a free option to withhold payment.

`CONFIRM_TIMEOUT` is therefore the most consequential parameter in §19, and it is a genuine trade-off rather than a tuning detail: too short and a buyer with a real complaint loses recourse by being slow; too long and every seller carries the float. It is stated here so the choice is made visibly rather than inherited from whatever a first implementation happened to pick.

Cancellation is not symmetric with acceptance. After `fulfilling` begins, a unilateral buyer cancellation is a *request*, not a transition — goods may already be in a courier's hands, and §8 custody has its own machine (§18.4). What the buyer can always do is refuse to confirm, which is what routes the trade to a dispute rather than to `closed`.

18.4 Consignment

Physical custody, which is the one machine whose transitions correspond to something happening in the world rather than a message being sent.

From	To	Trigger	Signed by	Timeout → behaviour
—	`created`	seller books a leg	seller	none
`created`	`accepted`	carrier or distributor takes custody	**the receiving party**	`PICKUP_TIMEOUT` → `created` again, so the seller re-books
`accepted`	`in_custody`	held at a hub	custodian	`HOLD_TIMEOUT` → alert; consolidation is waiting on a slower seller (§8.3)
`in_custody`	`handed_off`	passed to the next leg	**both** outgoing and incoming	none
`handed_off`	`delivered`	final handover	recipient, or carrier proof-of-delivery	`TRANSIT_TIMEOUT` → `lost`
any	`lost`	declared	current custodian, or by timeout	terminal

Custody transitions are signed by the party taking custody, not the party giving it up. A handoff attested only by the sender proves someone *tried* to hand something over; attested by the receiver, it proves the chain actually moved. This matters because the whole value of the chain is being able to say who held the goods when they went missing.

And it proves transfer, not recoverability. §8.4 says this and §18 has to repeat it here rather than let a tidy state machine imply otherwise: `lost` is a terminal state with a signed history and no remedy attached. Where the goods went is answerable; getting them back, or being paid for them, is §9's problem or nobody's.

18.5 Escrow

Only present when both parties chose an operator whose scope covers the trade (§9.4).

From	To	Trigger	Signed by	Timeout → behaviour
—	`funded`	buyer pays	operator attests	`FUND_TIMEOUT` → order `cancelled`
`funded`	`held`	seller dispatches	operator attests	—

From	To	Trigger	Signed by	Timeout → behaviour
`held`	`released`	buyer confirms, or order reaches `closed`	operator	`CONFIRM_TIMEOUT` (§18.3) → `released`
`held`	`refunded`	order reaches `cancelled`, or ruling	operator	—
`held`	`split`	ruling	operator	—
`held`	`held`	dispute raised	either party	`DISPUTE_TIMEOUT` → **see below**

The edge with no good answer. A dispute where neither party will move is exactly what escrow exists for, and it is also where non-custodial programmatic escrow deadlocks (§9.6). A custodial operator resolves it by ruling, which is why it is an operator class at all — and that ruling is published as a signed object (§9.5), so an operator that rules badly accumulates a record of it.

For a **non-custodial** rail there is no such move available. The honest options are a timeout that defaults to one party — which is a policy choice favouring whoever it defaults to, not a neutral mechanism — or indefinite lock. This section does not pretend a third option exists. What it requires is that the choice is **disclosed before the trade**, because a buyer who learns at dispute time that no one can release the funds was mis-sold the arrangement.

18.6 What every transition carries

- **Which party signs it.** A transition nobody signed is not evidence of anything.
- **What evidence it produces**, and whether that evidence is public (an escrow ruling, §9.5) or sealed (an order transition, §16.6).
- **Its timeout, and the state that timeout leads to.** Never "expires" alone: expiring *into* somewhere is the whole requirement.
- **Whether it is reversible**, and by whom. Most are not, which is why acceptance and confirmation are the two places a party gets to think.

18.7 Open

- **Whether `CONFIRM_TIMEOUT` is a protocol parameter or an offer-level term.** A seller of perishables and a seller of furniture want very different values, which argues for the offer; a buyer comparing offers should not have to read a timeout to know their recourse, which argues for the protocol. Currently leaning protocol floor with an offer-level extension upward only.
- **Whether a dispute pauses `CONFIRM_TIMEOUT` or runs alongside it.** Pausing lets a bad-faith buyer stall indefinitely by disputing; not pausing lets a slow operator time out a real dispute into an automatic release.
- **Whether `lost` needs a distinct `disputed-lost`**, given that custodian and recipient may disagree about whether delivery happened at all.

19. Parameters

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text

> becomes normative when the RFC 2119 keywords appear.

19.1 Why this is its own section

Parameters scattered through prose drift out of agreement with each other, and nobody notices until two sections cite different values for the same thing. Collecting them lets the linter check that a figure quoted in §N matches the table, and lets an implementer see the whole configuration surface at once instead of discovering it a section at a time.

The values below are **proposed defaults, not measurements**. Where a value is a genuine trade-off rather than an obvious floor, the trade-off is stated beside it — a number with no reasoning attached is a number the next person changes arbitrarily.

19.2 Order timeouts (§18.3)

Parameter	Proposed	Expires into	The trade-off
`PLACE_TIMEOUT`	72 h	`cancelled`	Long enough for a one-person seller to be away for a weekend; short enough that a buyer is not left guessing. A silent seller is never treated as having accepted.
`COUNTER_TIMEOUT`	72 h	`cancelled`	Symmetric with the above, since a counter puts the buyer in the seller's former position.
`FULFIL_TIMEOUT`	offer-declared	`cancelled`, escrow refunds	Cannot be one number: made-to-order declares lead days (§3), a download is instant. The availability axis already carries the expected duration, so this parameter is a *floor* on how long a buyer must wait before they may cancel — not the duration itself.
`CONFIRM_TIMEOUT`	**14 days**	**`closed`, escrow releases**	**The most consequential value in this table.** Too short and a slow buyer with a real complaint loses recourse by being slow. Too long and every seller carries the float on every order. 14 days is proposed because it sits near the cooling-off windows several consumer regimes use, which makes it defensible rather than arbitrary — but §11's research gap means that alignment is *asserted, not verified*.

19.3 Consignment timeouts (§18.4)

Parameter	Proposed	Expires into	Note
`PICKUP_TIMEOUT`	48 h	`created`	Returns to bookable rather than failing: a courier that never collected is a re-book, not a lost order.
`HOLD_TIMEOUT`	offer-declared	alert only	Consolidation waits on the slowest seller (§8.3). Raises an alert; does not cancel, because the buyer chose to wait.
`TRANSIT_TIMEOUT`	carrier estimate × 3	`lost`	Derived from the rate card's own published transit figure rather than a fixed number, since a same-city courier and a cross-border leg have nothing in common. The multiplier is the parameter; the estimate is the carrier's claim.

19.4 Escrow timeouts (§18.5)

Parameter	Proposed	Expires into	Note
`FUND_TIMEOUT`	24 h	order `cancelled`	An unfunded escrow holds nothing, so expiry costs nobody anything.
`DISPUTE_TIMEOUT`	operator-declared	operator ruling	**Custodial rails only.** A non-custodial rail has no ruling available, and §18.5 requires the resulting behaviour — default-to-one-party, or indefinite lock — to be disclosed before the trade rather than discovered at dispute time.

19.5 Object and serving limits

Parameter	Proposed	Rationale
`MAX_PRODUCT_RECORD`	64 KiB	A record describes a thing; images are separate content-addressed blobs.
`MAX_OFFER`	16 KiB	An offer is four axes and a few territories.
`MAX_RATE_CARD`	1 MiB	Zone tables are genuinely large. The one object where a big number is expected.
`MAX_REVIEW_BODY`	8 KiB	Also a **privacy** bound, not only a resource one: the smaller this is, the less room there is to type personal data into a public irrevocable object (§10.4).
`MIN_DIM_DIVISOR`	1000	Real carriers publish 5000 or 6000. A divisor between 1 and 1000 is not one any carrier uses, and it inflates the billable weight of every parcel computed from that card (§8.2). Zero is permitted and means “this carrier does not apply volumetric weight”.
`MAX_CART_LINES`	500	A cart is buyer-held (§6.1), so this bounds what a *seller* must be prepared to receive, not what a buyer may collect.

19.6 Rate and freshness

Parameter	Proposed	Rationale
`RATE_CARD_MAX_AGE`	30 days	Cards drift as surcharges change. Past this, a locally computed quote is stale enough to mislead, and the buyer is told rather than quietly given an old price.
`FEED_POLL_MIN`	60 s	A floor on how often a client re-fetches a seller’s feed, so an eager implementation does not become the load problem.
`GRANT_LIFETIME`	24 h	Analytics disclosure grants (§13) expire by default. A grant that never expires is consent that cannot be withdrawn by inaction.
`CLOCK_SKEW`	±5 min	Inherited from the substrate. Timestamps are for display and ordering; where order must be authoritative it comes from a feed sequence, never a clock (§16.7).

19.7 Open

- **Whether `CONFIRM_TIMEOUT` is a protocol floor or an offer-level term** (§18.7). Listed here too, because the answer decides whether this table is normative for it or merely suggests a default.
- **Whether `MAX_REVIEW_BODY` should be very much smaller.** Its privacy purpose argues for something nearer a sentence than eight kilobytes; its usefulness argues the other way. The current value was chosen for the second reason and should be revisited for the first.

- **Every value in §19.2 and §19.3 is a proposal with no measurement behind it.** They are plausible and internally consistent, and they have never been tested against how real sellers and couriers behave. Recorded as such rather than presented with confidence they have not earned.

20. References

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

20.1 Normative

Standards an implementation must read to be conformant.

Reference	Used for
RFC 2119 / RFC 8174	requirement language
RFC 8949	deterministic CBOR encoding
RFC 9052	COSE structured signatures
RFC 8032	Ed25519
RFC 5545	availability, recurrence (§3, §5)
ISO 3166-1	country codes (§8, §9, §11)
ISO 4217	currency codes (§5, §9)
DMTAP substrate	Identity, Feeds & Blobs, Sync, Roles, Wake (§0.3)

20.2 Informative

Reference	Relevance
schema.org product vocabulary	the §2 data model and existing-feed mapping
GS1 GTIN / SSCC	external identifier claims (§2), logistic units (§8)
Incoterms 2020	risk/cost transfer (§4, §8)
UN/CEFACT, EDIFACT	legacy order/despatch/invoice mapping (§7)
RFC 9458 (Oblivious HTTP)	IP-level unlinkability (§13)
UPU conventions	cross-border postal legs (§8)

20.3 On standards reuse

TRACT invents only where no standard exists. Where a proven specification fits, this document **profiles** it rather than restating it — and where it profiles, the referenced standard governs its own bytes.

21. Grounding — what the evidence actually supports

> **Why this section exists.** Every claim in this specification about what decentralized commerce can do is a claim about the future. The claims below are about the past, and they are less flattering. This section records what a 107-agent, adversarially-verified literature pass found in July 2026, including the findings that **contradict** parts of this document, so that a reader can tell the difference between a design decision and a demonstrated result.
>
> Nothing here is normative. Everything here should change what is normative.

21.1 Coverage — read this before citing anything below

Two passes have run. Between them, **four areas produced claims that survived three-vote adversarial verification:** deployed networks, global product identity, the goods/variant slice of the data model, and — from the second pass (§21.10) — live commerce state.

Still unresearched after both passes, and therefore neither supported nor refuted: logistics standards and consolidation optimisation (§8); privacy-preserving analytics and fraud (§13); and the whole of trust, dispute, tax and legal (§9.6, §10, §11). The second pass targeted all three directly and returned **nothing verified** on any of them, which makes them a standing gap rather than an oversight.

Those gaps are **absence of evidence, not evidence of absence**, and §8, §10, §11 and §13 must not cite this section as support. §11 in particular currently answers by assertion — who the marketplace facilitator is when there is no marketplace operator, and how erasure rights are satisfied against irrevocable objects, are both unanswered here. Within the covered areas, several named systems also produced nothing verified: Particl, Origin Protocol, BOLT12, x402, L402, and ActivityPub-based commerce.

21.2 The finding that most threatens this design

> **There is no deployed system achieving cross-publisher product identity without a licensed registry, and the one candidate for a permissionless alternative was refuted.**

The two deployed models are the only two:

Model	Property	Cost / gate
GS1 GTIN	permissioned monopoly namespace	*licensed*, not sold; issued through national member organisations; US$30 per identifier at GS1 US (verified 2026-07-21, US-specific — do not generalise). The free tier is explicitly restricted to closed/local circulation.
schema.org `productGroupID`	purely nominal	a plain `Text` string — no issuer, no registry, no uniqueness guarantee

There is nothing in between. A claim that crawl-derived clustering over schema.org identifiers demonstrates feasible permissionless product entity resolution was **refuted 0-3**. So this document contains **no positive evidence that cross-publisher product deduplication works at scale.**

What this means for §2. The content-address floor is sound as a *mechanism* — identical bytes converge, trivially. It is unproven as a *solution*, because independent publishers describing the same phys-

ical product do not produce identical bytes, and nothing here shows that the gap can be closed adversarially. §2's canonicalisation rules are therefore the load-bearing part of that section, not the addressing, and §2.5 must keep near-duplicate resolution listed as **open** rather than implied-solved. Language suggesting a global product view "falls out of" content addressing overstates what is known.

21.3 The OpenBazaar postmortem

OpenBazaar was the closest deployed relative of this design: signed objects, content-addressed listings, keypair identity, no operator. It shut down in January 2021. Cite it as a postmortem, never as a live comparison.

Measure	Result
Lifetime participants	~6,651
Concurrently online	~80
Credible sales, 14 months	~US$86,000
Median listing lifetime	~22 days
Share of measured sales value faked by one vendor	**60%**

Four failure modes, each of which maps onto a section of this document:

1. **Negligible economic activity** — orders of magnitude below centralized equivalents.
2. **Discovery re-centralized first.** A content-addressed substrate offers no global index, so search was outsourced to crawler operators, and the default one became a content-policy gatekeeper. → §2.6.
3. **Availability was bounded by publisher liveness.** Whole catalogues disappeared when a merchant node went offline; third-party indexes went stale against the live network. → §21.5.
4. **Reputation was trivially ballot-stuffed and structurally unenforceable**, and **opt-in escrow and moderation went unused precisely where they mattered most** — bad actors simply declined them. → §21.6.

Source caveat: all four rest on one peer-reviewed, DHS-funded measurement paper and share its methodology. Sales figures are an author-acknowledged lower bound from voluntary feedback, and item-survival numbers conflate deliberate delisting with liveness eviction.

21.4 What the largest live network chose instead

Beckn / ONDC is the biggest deployed "decentralized commerce" network, and it avoids OpenBazaar's failure modes by doing the opposite of this specification:

- **Registry-mediated, not self-certifying.** Participants register keys via `POST /subscribe`; counterparties resolve endpoints and public keys through a registry lookup — not gossip, not a DHT, not content addressing.
- **Approval-gated permissioned enrollment.** Portal whitelisting (6–48h), a signed agreement, certification, and probation before a participant may transact.
- **Identity anchored to DNS/TLS domain names, not keypairs** — inheriting CA and DNS centralization on top of registry centralization.

- **Discovery through operator-hosted, rate-limited lookup endpoints**, giving the operator a throttle and deny lever over network-wide discovery.

This is the honest comparison, and it is uncomfortable. The network with volume chose an operator at exactly the three points this document tries to leave operator-free. That is not proof this design fails; it is evidence that the burden of proof sits here, not with them.

Time sensitivity: Beckn developer docs are stamped 2021; ONDC has deprecated `/lookup` and `/vlookup` in favour of `/v2.0/lookup`, whose rate limit is published as "TBD".

21.5 Liveness is a first-class problem, not an edge case

§0.7 says durability lives at the edges and the sender's node retries. That is true **for orders** and false **for catalogues**.

A seller whose node is offline is not merely slow to receive orders — they are **invisible**. OpenBazaar's ~22-day median listing lifetime and mass catalogue disappearance on node departure are the measured consequence. §21.1 notwithstanding, this one is directly evidenced.

Implications this document must carry rather than bury:

- Documentation MUST NOT imply that an intermittently-online seller is fully functional. A laptop that sleeps can *receive orders* through the substrate's mailbox and wake path; it cannot *serve a catalogue* while asleep.
- Third-party caching/pinning of public catalogue objects moves from a nice-to-have to a requirement for any seller not running an always-on node — and **unpaid replication is exactly what OpenBazaar did not get**. Whether pinning needs an incentive, and whether that incentive creates another operator, is open.
- A gateway serving a store is, incidentally, a liveness backstop. That is an argument *for* the operator class of §0.4.2, and it should be counted honestly as one.

21.6 Reputation: the documented failure mode is this design's

OpenBazaar's reputation failure was **self-published, unweighted reviews with no banning authority**, combined with **opt-in escrow that bad actors declined**. One vendor faked 60% of measured sales value.

§10 proposes purchase-attested reviews, which is strictly stronger than what OpenBazaar had — an attestation binds a review to a transaction. But two of OpenBazaar's failure conditions survive unchanged in this document as currently written:

1. **Self-dealing.** A seller transacting with itself produces genuine attestations. Attestation raises cost; it does not establish that the counterparty was independent.
2. **Opt-in escrow is declined by exactly the actors it exists to constrain.** §9.6 makes escrow per-order and optional, and §9.6's own reasoning — that mandatory escrow would exclude underserved regions — is sound. But the measured consequence of optionality is that it goes unused where it matters. Both cannot be true costlessly.

§10 and §9.6 must state this rather than claiming attestation resolves it. The sybil-cost floor achievable on a signed-feed substrate is listed in §21.8 as an open question because the attack literature was not reached by this pass.

21.7 A warning about scope

Nostr — a signed-feed ecosystem structurally similar to this substrate — marked its marketplace specification **NIP-15 "unrecommended: too complicated"** in-repo (banner added 2026-05-31) and redirected implementers to NIP-99, a far simpler classified-listing primitive. Neither solves "many stores, one SKU": product IDs are merchant-generated and merchant-scoped, discovery is free-text tags, and there is no cross-publisher product identifier.

NIP-15 also moves the entire order and checkout flow off the public feed into encrypted messages with **the merchant as sole authority** asserting payment and shipment — no escrow, no buyer counter-signature, no third-party-verifiable order state. This document's §7 signed-transition model is a deliberate departure from that, and the departure is the point; but the ecosystem's own verdict on structured-commerce complexity is a caution this specification should not ignore.

Wording note: the repo status word is "unrecommended", not "deprecated".

21.8 Open questions this pass could not close

1. **Does any deployed system achieve cross-publisher product identity without a licensed registry?** The design space between GS1 monopoly and nominal merchant string is currently evidence-free. Needs a dedicated pass on entity resolution, blocking strategies and probabilistic matching **under adversarial publishers** — including deliberate collision.
2. **What is the achievable sybil-cost floor for reputation on a signed-feed substrate?** Do buyer counter-signatures, transaction-cost binding, or local-only web-of-trust scoring actually resist ballot-stuffing? Entirely unresearched.
3. **Where does a spec put the operator-shaped functions it cannot eliminate — indexer, registrar, arbiter, pinner?** ONDC answers "one central approval-gating registry". OpenBazaar answered "nowhere" and got a gatekeeping default search engine plus liveness-bounded availability. **Is there a middle design** — multiple competing indexers with verifiable completeness or censorship proofs, federated or rotating registrars, paid pinning markets — **with any deployed precedent and measured outcomes?** This is the central unanswered question for §0.4.
4. **How do erasure rights and identifiable-seller mandates interact with immutable, content-addressed, replicated commerce objects?** Who is the data controller when order data is replicated across independent nodes; how is erasure satisfied against irrevocable objects; who satisfies trader-traceability duties when identity is a keypair; and who is the "marketplace facilitator" for tax purposes when no marketplace operator exists. **These are the most likely hard blockers on a no-operator design**, and §0.5.1 and §11 currently answer them by construction and by assertion respectively, not by evidence.

21.9 What this section obliges

- §2 must not imply that content addressing solves global product identity (§21.2).
- §2.6 must state that "any node MAY build an index" does not prevent one index becoming the de facto gatekeeper (§21.3, §21.4).
- §0.7 and all self-hosting guidance must distinguish receiving orders from serving a catalogue, and must not present an intermittently-online seller as fully functional (§21.5).
- §9.6 and §10 must state the opt-in-escrow and self-dealing failure modes as measured outcomes, not hypotheticals (§21.6).

- No section may cite this one as support for logistics, analytics, live-state or legal claims (§21.1).

21.10 Second pass — live commerce state (July 2026)

A second adversarially-verified pass covered the four areas §21.1 records as unresearched. **Only one of them produced surviving claims: live commerce state.** Logistics and delivery, privacy- preserving analytics and fraud, and the whole of trust/dispute/tax/legal produced **no verified findings and remain unresearched** — §21.1's prohibition on citing this section for them still stands, and §8, §10, §11 and §13 are still unevidenced.

21.10.1 The good news: no-coordinator oversell prevention is real

A seller running N replicas **can** guarantee no oversell with no coordinator. The mechanism is the escrow/demarcation family — O'Neil (1986) → Barbará-Millá & Garcia-Molina (1994) → Balegas et al., SRDS 2015 (`BoundedCounter`), shipped in production as AntidoteDB's `antidote_crdt_counter_b`.

The gap between the counter and its bound is pre-partitioned into transferable **rights**. A replica may decrement only from rights it holds locally, and the locally computed right count is always conservative — it can under-count but never over-count. So the invariant holds **even from stale state**, and the guarantee survives arbitrary message loss, delay, reorder and partition.

The contrast matters: a plain eventually-consistent read-check-decrement counter measurably **does** oversell — roughly 200 excess decrements at ~200 concurrent clients in the SRDS'15 experiment. This is not a theoretical risk being designed around.

The guarantee is safety-only and is paid for entirely in liveness: stranded quota, spurious "sold out", and aborted decrements while global stock remains. §6.4 already states that residual.

21.10.2 The bad news: four operator-shaped roles inside the counter

Each of these must be accepted or replaced explicitly by §6. None was visible before this pass.

1. **An intra-replica serialization point is mandatory.** A single-writer node, a compare-and-swap, or consensus. **A "replica" therefore cannot itself be a set of mutually uncoordinated nodes** — the coordination is not removed, it is pushed down one level and made local.
2. **The failure model is crash-recovery with durable state, not Byzantine.** This is the sharpest finding in the pass. **A lying or state-losing replica simply oversells**, and none of this literature covers stock held across *mutually distrusting* parties. The mechanism is sound for a seller's own replicas, which trust each other; it says nothing about a seller who lies.
3. **Reclaiming a permanently-dead replica's rights requires an agreement or failure-detector decision** — and if that decision is wrong, rights are double-issued and the invariant breaks. Deciding a peer is dead is exactly the kind of judgement a coordinator makes.
4. **With three or more replicas, transfer is no longer anonymous.** The giver must name the recipient, which reintroduces an allocation policy — and the papers themselves suggest centralising it.

21.10.3 Sagas do not provide what multi-party checkout needs

Confirmed rather than assumed: sagas explicitly provide **neither atomicity nor isolation**. Other parties observe partial sagas, and there is no abort cascade. So multi-party checkout gets eventual amendment, never oversell prevention — **oversell prevention has to sit in the per-replica invariant**

primitive, not in the checkout flow above it. §6.4's "no cross-seller atomicity" is therefore not a limitation to be engineered away later; it is the correct shape.

21.10.4 What §6 must now say

- Name the intra-replica serialization requirement rather than implying replicas need no coordination at any level (§21.10.2 item 1).
- State the **non-Byzantine failure model as a hard boundary**: bounded counters protect a seller from their own concurrency, not a buyer from a dishonest seller. Conflating the two would be a safety claim the literature does not support (item 2).
- Specify dead-replica right reclamation as an explicit, fallible decision with a stated failure mode, not a background detail (item 3).
- Acknowledge that transfer names a recipient at $N \geq 3$, and say who chooses (item 4).

Drafted with the help of an LLM by Imran Yusuf Paruk · Durban, South Africa